



C1

Inside the model

~45 min · plain-language first · FOR-BUILDERS layers optional

BY THE END OF THIS CAPSULE YOU WILL BE ABLE TO

Explain in plain words how a large language model turns text into an answer — tokens, next-token prediction, attention, training and scale — and say why the same machine that is brilliant also makes things up.

From black box to open box

Week 2 you rented the model through an API. Today we open it — five questions, one engine underneath.

- **1 • Token and embedding** — the piece of text, and the numbers that stand for it
- **2 • What does it DO?** — predict the next token — that's the whole engine
- **3 • How does it read context?** — attention — it looks back at what matters
- **4 • How is it trained?** — pre-train → instruct → align (base model → assistant)
- **5 • Why smart, why wrong?** — scale & emergence — and why it hallucinates

Takeaway Five questions today; everything reduces to “predict the next token.”

1 • Tokens — the model's alphabet

The model never sees words or letters — only tokens turned into numbers.

You type	Model sees (tokens)	Note
cat	[cat] → 1 token	common word = 1 piece
unbelievable	[un][believ][able] → 3	rare words split
草莓 (“strawberry”)	[草][莓] → 2	non-English often costs more tokens
GPT-4o	[G][PT][-][4][o] → 5	odd strings fragment

For builders: more tokens in = more compute = more cost (a Capsule-3 theme).

Takeaway Everything in and out is tokens — half the model's quirks follow from that.

How many letter “r”s can the model actually “see” in “strawberry”?

A · Three

B · Two

C · One

D · It can't see letters at all

Guess first — then we run the tokeniser live (Class-1 notebook, pre-run fallback ready).

Under the hood — sub-word tokenisation

FOR BUILDERS

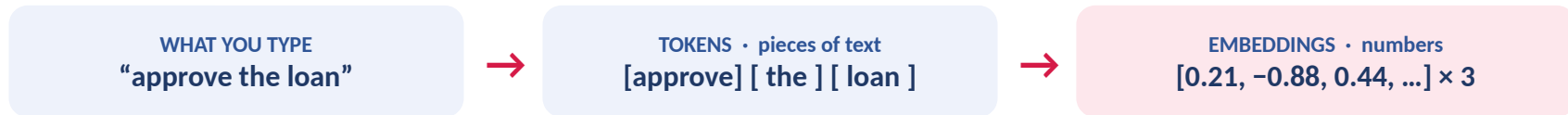
How text becomes numbers the network can compute on.

- **Byte-Pair Encoding (BPE)** — start from characters, repeatedly merge the most frequent pair → a fixed vocabulary (~100k tokens)
- **Token → ID → embedding** — each token maps to an ID, then to a learned vector; similar meanings sit near each other
- **Why it matters** — vocabulary choice sets cost, speed, and how well the model handles code, maths and non-English text

Takeaway Tokenisation is invisible but sets cost and capability — it is a design choice, not a detail.

From token to embedding — and which “embedding” we mean

The model never sees your words. It sees one list of numbers per token — and that is not the same thing you built in Week 2.



One vector per token — and they do not stay put: attention reshapes them layer by layer, so “bank” starts identical in “river bank” and “bank loan” and ends up in two different places.

	TODAY (C1) • token embeddings	WEEK 2 (RAG) • text embeddings
One vector per...	a TOKEN — a piece of a word	a CHUNK of text — a paragraph or a page
Produced by...	a lookup table inside the model’s own weights	a separate embedding model you call
Where it lives...	inside the model — never stored	in a vector database you own
Its job...	let the network do maths on meaning	find the passage closest to the question

Takeaway Same idea — meaning as coordinates. Different granularity, different model, different job.

2 • It predicts the next token

Given the text so far, it scores every possible next token, picks one, appends it, and repeats.

Example — “The cat sat on the ____”

The model outputs a probability for every token in its vocabulary:

mat 61% • floor 12% • rug 7% • sofa 4% • roof 0.3% ...

It samples one, appends it, and predicts again. That loop is the entire engine.

- **No lookup, no plan** — it isn't retrieving facts or “thinking ahead” — just predicting the next piece
- **Brilliance & error, one source** — fluent reasoning and confident nonsense come from the same operation

Takeaway “Autocomplete on steroids” — everything else today explains how this gets so good, and where it breaks.

Which of these does the model NOT actually do?

A • Look words up in a database

B • Reason step-by-step by default

C • Predict the next token

D • Both A and B

Keep your pick — we test it against how the engine really works.

Under the hood — how it “chooses”

FOR BUILDERS

Same model, same prompt — different knobs give different answers.

Knob	Low	High
Temperature	safe, repeatable, “boring”	creative, varied, riskier
Top-p (nucleus)	pick only from the most-likely tokens	allow a longer tail of options

Set temperature = 0 for reproducible runs — important when you evaluate prompts (Capsule 2).

Takeaway Sampling explains why the “same” prompt can vary — and why evals fix the knobs first.

3 • Attention — it looks back at what matters

For each new token, the model weighs every earlier token and pulls in the relevant ones.

Example

“The trophy didn’t fit in the suitcase because it was too big.”

To predict what “it” refers to, attention links “it” → “trophy” (not “suitcase”).

That linking, done for every token, is how the model tracks meaning across a passage.

- **Plain version** — like re-reading a sentence and highlighting the words that matter for the next one
- **Why you care** — it’s why giving the model the right context (your Week-2 RAG) changes the answer

Takeaway Attention = the mechanism that lets earlier words shape the next one.

Under the hood — the Transformer

FOR BUILDERS

What “attention” is actually computing.

- **Query, Key, Value (QKV)** — each token asks a Query, matches every token’s Key → weights, then mixes their Values
- **Multi-head** — many attention heads run in parallel, each learning a different relationship
- **Layers** — stack the block dozens of times; meaning builds up layer by layer
- **Context window** — the max tokens it can attend to at once — the model’s “working memory”

Takeaway One idea — weighted look-back (QKV), repeated in parallel and stacked — is the whole Transformer.

Why did giving the model more context make your Week-2 RAG better?

Discuss in pairs — 2 minutes.

Hint: think about what attention can and cannot reach.

We'll take two answers, then move on.

4 • How it's trained — three stages

A raw text-predictor becomes a helpful assistant in three steps.

Stage	What it learns	Result
1 • Pre-train	autocomplete the internet	language + world knowledge (a raw “base” model)
2 • Instruct / SFT	follow instructions from examples	does what you ask, not just continue text
3 • RLHF	align to human preference	helpful, honest, safer answers

SFT = supervised fine-tuning • RLHF = reinforcement learning from human feedback.

Takeaway Base model = raw autocomplete; the assistant you talk to is stages 2–3 on top.

Base vs instruct — which do you call from an API, and why?

A · The base (pre-trained) model

B · The instruct model

C · Doesn't matter

Ties back to Class-2 model choice (open vs closed, base vs instruct).

5 • Scale changed everything

More data + compute + parameters → predictably better. And new abilities appear that no one trained.

- **Scaling laws** — loss falls smoothly as you add data, compute and parameters — performance is predictable enough to plan billion-dollar training runs
- **Emergence** — abilities that are absent in small models appear past a scale threshold — translation, coding, in-context learning
- **So what** — you can't always tell in advance what a bigger model will be able to do — capability is discovered, not fully designed

Takeaway Scale didn't just improve the model — it unlocked abilities nobody explicitly built.

Which of these is an EMERGENT ability?

A · Repeating text from its training data

B · Translating a language pair it was never shown together

C · Storing every fact in a lookup table

Emergent = appeared with scale, not directly trained.

Why it makes things up

The same machine that is brilliant also fabricates — for one simple reason.

Same mechanism, two faces

A brilliant answer and a confident fabrication are produced the identical way:

the model predicts a plausible next token — whether or not it “knows.”

It has no built-in “I’m not sure.”

- **Not lying** — no intent — just the most likely continuation, which is sometimes wrong
- **Why it matters** — fluent + confident + wrong is the dangerous combination — you must measure outputs (C2), especially in high-stakes work (C3)

Takeaway Brilliance and hallucination come from the same mechanism — which is why we measure.

Where this bites — three industries

The mechanism is neutral; the stakes are yours.

Industry	A confident, silent error looks like...	Why it's hard to catch
Healthcare	a plausible but wrong dosage or contraindication	reads as fluent clinical text
Banking / finance	a fabricated policy clause or mis-stated number	sits inside an otherwise correct answer
Legal	a citation to a case that doesn't exist	formatted exactly like a real one

Takeaway Same next-token engine everywhere — what changes is the cost of being wrong.

Where would a confident, silent error hurt most in YOUR work?

1 min think · 1 min share with a neighbour.

And: how would anyone even notice it happened?

We'll collect 3-4 across sectors before the break.

CAPSULE 1 • TAKEAWAY

It predicts the next token.

Brilliance and hallucination come from the same mechanism. Next (Capsule 2): steer it with prompting — and prove it with evals.